Proprietăți și tipuri de clase

Compoziție (Composition)

Compoziția este o relație puternică de tip *„parte-întreg”*, în care obiectele componente nu pot exista independent de obiectul principal.

- Dacă obiectul „container” este distrus, și componentele sale sunt distruse.

Exemplu:

```
class Motor {
    void porneste() {
        System.out.println("Motor pornit");
    }
}

class Masina {
    private Motor motor;

    public Masina() {
        motor = new Motor(); // creat intern
    }

    void pornesteMasina() {
        motor.porneste();
    }
}
```

Motor nu are sens fără Masina → **compoziție**

Agregare (Aggregation)

Agregarea este o relație mai slabă, în care obiectele pot exista independent unele de altele.

- Obiectul „container” NU controlează ciclul de viață al componentelor.

Exemplu:

```
class Profesor {
    String nume;
}

class Universitate {
    private Profesor profesor;

    public Universitate(Profesor profesor) {
        this.profesor = profesor;
    }
}
```

Profesor poate exista fără Universitate → **agregare**

Diferențe între compoziție și agregare

Caracteristică	Compoziție	Agregare
Relație	puternică	slabă
Ciclu de viață	dependent	independent
Creare obiect	intern	extern
Exemplu	Mașină–Motor	Universitate–Profesor

Șiruri de caractere (Strings)

În Java, șirurile de caractere sunt obiecte din clasa String.

String text = "Salut lume!";

Caracteristici:

- sunt **imutabile**
- sunt frecvent utilizate
- sunt optimizate prin *string pool*

Clasa String

Clasa String este una dintre cele mai utilizate clase din Java.

Proprietăți:

- imutabilă
- stocată în heap
- suportă literal strings

Metode importante:

String s = "Java";

Metodă	Descriere
length()	lungimea
charAt(i)	caracter la poziția i
substring()	extrage subșir
toUpperCase()	uppercase
equals()	comparare conținut
contains()	verifică subșir

Exemplu:

```
String s = "Programare";

System.out.println(s.length());
System.out.println(s.substring(0, 5));
System.out.println(s.toUpperCase());
```

Regex-uri în Java

Regex (Regular Expressions) sunt folosite pentru validarea și manipularea textului.

Clase principale:

- Pattern
- Matcher

Exemplu:

```
import java.util.regex.*;

Pattern p = Pattern.compile("\\d+");
Matcher m = p.matcher("123abc");

if (m.find()) {
    System.out.println("Numar gasit");
}
```

Reguli de bază:

Regex	Semnificație
.	orice caracter
\\d	cifră
\\w	caracter alfanumeric
+	una sau mai multe apariții
*	zero sau mai multe
^	început
$	sfârșit

Exemplu validare email:

String regex = "^[\\w.-]+@[\\w.-]+\\.\\w+$";

Clasa StringBuilder

Mutabilă și eficientă pentru concatenări.

Exemplu:

```
StringBuilder sb = new StringBuilder("Salut");
sb.append(" lume");
System.out.println(sb);
```

Avantaj:

- performantă (nu creează obiecte noi)

Clasa StringBuffer

Similară cu StringBuilder, dar este **thread-safe** (sincronizată).

Exemplu:

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" Buffer");
```

Diferență:

Clasă	Thread-safe	Performanță
StringBuilder	nu	mai rapid
StringBuffer	da	mai lent

Clase imutabile

O clasă imutabilă nu își poate modifica starea după creare.

Exemple:

- String
- Integer

Cum creezi una:

```
final class Persoana {
    private final String nume;

    public Persoana(String nume) {
        this.nume = nume;
    }

    public String getNume() {
        return nume;
    }
}
```

reguli:

- final class
- atribute final
- fără setter

Clase record

Introduse în Java pentru modele simple de date.

Exemplu:

```
record Student(String nume, int varsta) {}
```

Automat generează:

- constructor
- getters
- equals(), hashCode(), toString()

Interfețe (Interfaces)

O interfață definește comportament, nu implementare.

Exemplu:

```
interface Animal {
    void sunet();
}
```

Implementare:

```
class Caine implements Animal {
    public void sunet() {
        System.out.println("Ham");
    }
}
```

Extinderea interfețelor

O interfață poate extinde alta.

```
interface A {
    void metodaA();
}

interface B extends A {
    void metodaB();
}
```

Utilizarea interfețelor

Beneficii:

- suport pentru **polimorfism**
- separare între logică și implementare
- flexibilitate

Exemplu:

```
interface Forma {
    double arie();
}

class Cerc implements Forma {
    double raza;

    Cerc(double raza) {
        this.raza = raza;
    }

    public double arie() {
        return Math.PI * raza * raza;
    }
}
```

Utilizare:

```
Forma f = new Cerc(5);
System.out.println(f.arie());
```

Exerciții

1. Biblioteca

- Creează o clasă **Book**:
  - câmpuri: titlu, autor, ISBN
  - ISBN trebuie să fie **imutabil**
- Creează o clasă **Author**:
  - câmpuri: nume, prenume
- **Relații:**
  - Biblioteca conține o listă de **Book**
  - **Book** are un **Author**
- **Folosire record:**
  - Creează un record pentru Imprumutul unei cărți care să conține numele clientului și al cărții
- Afișează toate cărțile:
  - titlul cu toUpperCase()
  - autorul formatat: „Prenume NUME”
- Verifică dacă ISBN:
  - are exact 13 caractere
  - conține doar cifre

2. Validarea datelor unui utilizator

- **Creează o clasă Utilizator:**
- câmpuri:
  - email

    - parola
- clasa trebuie să fie **imutabilă**
- Email:
    - format valid (ex: test@mail.com)
- Parola:
    - minim 8 caractere
    - cel puțin o cifră
    - cel puțin o literă mare
- **Creează o clasă Validator:**
    - metodă:

boolean valideazaEmail(String email)
boolean valideazaParola(String parola)

- Creează mai multe obiecte Utilizator
- Afișează rezultate validare
- Extrage domeniul emailului (substring)
- Verifică dacă parola conține un anumit caracter

3. Text Editor

- **Creează două clase:**
    - EditorRapid → folosește StringBuilder
    - EditorSigur → folosește StringBuffer
- adăugare text (append)
- ștergere text
- inserare text
- construiește un text lung (minim 100 concatenări)
- compară performanța între cele două
- implementează metodă:
    - int numarCuvinte()

4. Forme Geometrice

- **Creează interfața:**

```
interface Forma {
   double arie();
}
```

- **Creează interfață extinsă:**

```
interface FormaColorata extends Forma {
   String culoare();
}
```

- **Clase implementate:**
  - Cerc
  - Dreptunghi
  - Triunghi
- Creează o listă de tip Forma
- Adaugă obiecte diferite
- Apelează arie() pentru fiecare
- afișează descriere:
  - "Cerc de culoare ROSU cu aria: ..."
- folosește instanceof pentru verificări
- folosește toString() personalizat